

The Legal Agent Project

A plain-English account of what was built, what was found, and what is still missing

Section 138 Research Assistant · Supreme Court of India cheque-dishonour precedent · Report generated 18 August 2026

About the tool this report describes. It is an information tool, not legal advice. It cannot represent anyone, does not know the facts of a case beyond what is typed into it, and can be wrong. Deadlines under Section 138 are short and missing one can end a claim. Anyone relying on it should consult an advocate.

1. What we set out to build

The goal was a tool for ordinary people — not lawyers — who have been given a cheque that bounced. Someone in that position has a real legal remedy under **Section 138 of the Negotiable Instruments Act 1881**, but it is hedged with short deadlines that are easy to miss and fatal to a claim once missed. Most people do not know those deadlines exist.

The tool was to do two things: search past Supreme Court judgments about cheque dishonour and explain what they say, and work out where the person stands on the statutory clock.

It was modelled on an existing working system — the GeM tender search portal — which already had a proven pipeline: download documents, read them, convert them into searchable mathematical form, and search them by meaning as well as by keyword.

2. The problem that shaped everything: getting the judgments

The hardest part of this project was never the artificial intelligence. It was obtaining the judgments in the first place.

Indian court judgments are public documents, but they are not made available for bulk automated download. Three sources were checked directly rather than assumed:

- **digiscr.sci.gov.in** — a web address quoted from memory during planning. It **does not exist**. This was an early and useful reminder: anything recalled from memory rather than verified is unreliable, and in a legal tool that is dangerous.
- **judgments.ecourts.gov.in** — the official portal covering the Supreme Court and all High Courts. It is protected by an **image CAPTCHA**, the puzzle that asks you to prove you are human.
- **sci.gov.in** — the Supreme Court's own site. Its judgment search is also CAPTCHA-protected.

A CAPTCHA is a deliberate instruction not to automate access. No attempt was made to defeat it. That decision closed the obvious route and forced a better one.

3. The source that was used instead

A published research dataset was found that already contains this material, collected lawfully and released for reuse: **28,470 Supreme Court judgments**, licensed CC-BY-4.0, which permits reuse with attribution.

Crucially, its origin is verifiable from the data itself rather than taken on trust. Every record carries an internal reference number beginning **ESCR** — the Supreme Court's own electronic reports system. The

judgments genuinely come from the Court; somebody else did the CAPTCHA-constrained collection work and published the result.

Two further points on the legal position. Judicial pronouncements are exempt from copyright under **section 52(1)(q) of the Copyright Act 1957**, so the judgments themselves are free to use. The CC-BY-4.0 licence covers the compilation effort, and the required attribution is recorded permanently in the project's manifest file.

4. A decision that saved a third of the work

The dataset covers 1950 to 2025 — 76 years, about 121 gigabytes in full.

Not all of it is relevant. **Section 138 did not exist before 1989**. It was inserted into the Act by an amendment that came into force on 1 April 1989. Judgments before that date are structurally incapable of containing Section 138 precedent.

So 39 of the 76 years were excluded — not as a shortcut, but as a matter of law. Only 1989 onwards is downloaded.

What the data then revealed. The years 1989 to 1992 returned **zero** Section 138 cases. This is not a fault. The section commenced in 1989 and cases take years to work their way up to the Supreme Court, so the earliest precedent realistically appears in the mid-1990s. The legal reasoning was sound; the practical starting point is simply later.

5. What was actually built

Six programs, each doing one job, in sequence.

`escr_ingest.py` — the collector

Downloads the judgments year by year. It keeps a manifest recording what has already been fetched, so re-running it collects only what is missing rather than starting again.

`build_corpus.py` — the reader and filter

Judgments arrive as PDFs. This extracts the text and then decides which judgments are genuinely about Section 138.

That filter took two attempts, and the difference matters. The first version accepted any judgment mentioning both "Negotiable Instruments" and "section 138". Checking the results showed that a third of them never used the word "cheque" at all — they were about something else entirely and had merely cited Section 138 in passing.

Reasoning from a case that is not really about your problem is worse than returning nothing. So the filter was tightened: a judgment must now refer to Section 138 repeatedly *and* use the vocabulary a real cheque case necessarily uses — drawer, payee, dishonour, demand notice. On 2024's judgments this cut the count from 31 to 17, rejecting 45% as not genuinely on point.

The surviving judgments are then cut into overlapping passages. Judgments range from 15,000 to 376,000 characters, so treating one as a single searchable unit would be meaningless. The passages overlap deliberately, because a legal proposition and the authority supporting it often sit in adjacent paragraphs and a clean cut between them would leave both halves unintelligible.

`embed_corpus.py` — the translator

Converts each passage into a list of numbers positioned so that passages meaning similar things end

up close together. This is what allows someone to search in ordinary English and find a judgment that uses entirely different words.

s138_rules.py — the deadline calculator

This is the part with no artificial intelligence in it whatsoever, and that is the point.

Section 138 is a chain of deadlines, and deadlines are arithmetic. Arithmetic must never be delegated to something that can invent an answer. This program applies the statute directly:

- A cheque is valid for **three months** from its date and must be deposited within that time.
- Once the bank reports the bounce, a written demand notice must reach the drawer within **30 days** — Section 138 proviso (b).
- The drawer then has **15 days** to pay — proviso (c). If they pay in full, no offence remains.
- If they do not, a complaint may be filed within **one month** of that window closing — Section 142(1)(b). Filing before that date is premature and liable to be dismissed.

It also checks the ingredients of the offence: the cheque must be from the drawer's own account, for a debt actually owed, and returned for insufficient funds. Each result cites the exact provision it comes from.

retrieval.py — the search engine

Runs two different searches and merges them. One searches by meaning; the other searches for exact words, which matters because "Section 139" or a case citation are literal strings that meaning-based search mangles.

Its most important feature is the ability to **refuse**. If neither search finds anything well supported, it returns nothing rather than the least-bad match.

app.py — the screen you use

A web page with two tabs. **Research a question** takes a plain-English description and returns real judgment passages, each labelled with its citation, the judges, and the date. **Check my deadlines** takes your dates and computes exactly where you stand, then automatically searches for precedent on whichever requirement is weakest in your particular case.

When the search finds nothing, the page says so plainly and points to the District Legal Services Authority, which provides free legal aid.

6. Two bugs found and fixed

The refusal system was completely broken. This was the most serious problem found, because refusing to answer is the single most important safety feature in the design.

Testing with the nonsense query *"recipe for chicken biryani"* returned three confident results from Supreme Court judgments. The tool would have answered literally anything.

The cause was that the confidence thresholds had been *guessed* rather than measured. The cut-off was set at a distance of 1.20, but no query this system produces ever reaches that number, so every query — however absurd — scored maximum confidence.

The fix was to measure. Genuine legal questions produced distances between **0.41 and 0.68**. Off-topic questions never came closer than **0.88**. The threshold was moved to 0.80, inside that gap. All five legal

test questions now answer; all five nonsense questions are correctly refused.

The downloader could not resume. A network timeout at 189 megabytes of a 203-megabyte file threw away the entire download and started again. Over roughly ten gigabytes of transfers this would have been crippling. It now resumes from where it stopped.

Why this matters. Both bugs were found by deliberately testing for failure rather than confirming success. The refusal bug in particular would never have appeared in normal use — every sensible question returned sensible answers. It only surfaced because the system was asked something absurd.

7. Where the project stands today

- Supreme Court judgments available, 1989–2025: **28,470**
- Years of full text downloaded so far: **8 of 37** (about 1.6 GB; the rest is still running)
- Judgments identified as genuinely about Section 138: **17**
- Searchable passages indexed: **231**
- PDF text extraction success rate: **782 of 782, no failures**
- Expected final size, based on the measured 4% rate: roughly **1,100 judgments**

The system works from end to end. Its coverage is thin only because the download is incomplete, and it grows automatically as the remaining years arrive.

8. What is deliberately missing

- **No AI writes any legal content.** Everything shown is text taken word-for-word from a real judgment. A plain-language layer that rephrases those passages into simpler English is planned as the last step, and will be strictly limited to rewording retrieved text. It will never be allowed to produce a case name or a legal proposition of its own, because that is precisely how AI legal tools have produced fabricated citations that lawyers have been sanctioned for filing.
- **Supreme Court judgments only.** High Court judgments on Section 138 are far more numerous and often more directly applicable to a person's local court.
- **No check on whether a judgment is still good law.** Judgments can be overruled or overtaken by amendments to the Act. The tool does not yet detect this, and it is the most important outstanding gap.

9. The one thing that must be verified by a lawyer

The statutory periods encoded in the deadline calculator — the three-month cheque validity, the 30-day notice window, the 15-day payment period, the one-month filing window — were taken from the Act as understood during the build. They have not been confirmed by a qualified advocate.

A wrong number there produces confidently wrong deadlines, delivered to someone with no way of knowing they are wrong. That is the worst possible failure this tool could have, and it is the one thing that should be checked before anybody relies on it.